

LangChain Field Notes — Prompts | Memory | Chains | Runnables

SECTION 0 : PROMPTS

PromptTemplate Creates a single text prompt with placeholders. Used when you want a final string instead of chat messages. Main attributes are template, input_variables, partial_variables, examples, example_prompt, prefix, suffix, and example_separator. LangChain validates and formats these placeholders before sending the final prompt to the model. <pre>from langchain_core.prompts import PromptTemplate prompt = PromptTemplate(template="Explain {topic} in {level} language.", input_variables=["topic", "level"], partial_variables={"level": "beginner-friendly"}) final_prompt = prompt.format(topic="LangChain") print(final_prompt)</pre>	ChatPromptTemplate Used mainly with chat models such as ChatGPT, Groq, Gemini, or Claude. It separates instructions into messages such as system, human, and ai. - system: Defines the assistant's behavior. - human: Contains the user's question. - ai: Can represent an earlier assistant response. <pre>from langchain_core.prompts import ChatPromptTemplate chat_prompt = ChatPromptTemplate.from_messages([("system", "You are a helpful {role}."), ("human", "{question}")]) messages = chat_prompt.format_messages(role="Python teacher", question="What is a dictionary?") for message in messages: print(message.type, ":", message.content)</pre>	MessagesPlaceholder A placeholder inside ChatPromptTemplate for inserting dynamic conversation history. For example, it can insert previous user and AI messages before the current question. <pre>from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder from langchain_core.messages import HumanMessage, AIMessage chat_prompt = ChatPromptTemplate.from_messages([("system", "You are a helpful AI assistant."), MessagesPlaceholder(variable_name="chat_history"), ("human", "{question}")]) messages = chat_prompt.format_messages(chat_history=[HumanMessage(content="My name is Somnath."), AIMessage(content="Nice to meet you, Somnath.")], question="What is my name?") for message in messages: print(message.type, ":", message.content)</pre>
--	--	--

FewShotPromptTemplate

Used when you want to provide examples of the expected input and output before asking the real question.
This helps the LLM follow a specific answer format, tone, or task pattern.

```
from langchain_core.prompts import PromptTemplate,  
FewShotPromptTemplate
```

```
examples = [  
    {  
        "question": "What is Python?",  
        "answer": "Python is a programming language."  
    },  
    {  
        "question": "What is LangChain?",  
        "answer": "LangChain is a framework for building LLM apps."  
    }  
]
```

```
example_prompt = PromptTemplate(  
    template="Question: {question}\nAnswer: {answer}",  
    input_variables=["question", "answer"]  
)
```

```
few_shot_prompt = FewShotPromptTemplate(  
    examples=examples,  
    example_prompt=example_prompt,  
    prefix="Answer each question in one short sentence.",  
    suffix="Question: {question}\nAnswer:",  
    input_variables=["question"],  
    example_separator="\n\n"  
)
```

```
final_prompt = few_shot_prompt.format(  
    question="What is an LLM?"  
)  
print(final_prompt)
```

SECTION 1 : MEMORY

LangChain memory lets an LLM application remember information from earlier interactions. Without memory, every new question is independent—the model does not know what the user said previously.

Main Components

Session ID / Thread ID	A unique ID for one user conversation. It prevents one user's messages being mixed with another user's history.
<pre>session_id = "somnath_chat_1" store = {} if session_id not in store: store[session_id] = InMemoryChatMessageHistory()</pre>	
Memory store / Checkpointer	The place where history is saved—memory, SQLite, PostgreSQL, Redis, etc.
<pre>from langchain_core.chat_history import InMemoryChatMessageHistory history = InMemoryChatMessageHistory() history.add_user_message("My name is Somnath.") history.add_ai_message("Nice to meet you, Somnath.") print(history.messages)</pre>	
RunnableWithMessageHistory	A LangChain wrapper that automatically: • Reads old chat history. • Sends it to the prompt. • Saves the latest user question and AI response.
<pre>from langchain_core.runnables.history import RunnableWithMessageHistory memory_chain = RunnableWithMessageHistory(chain, get_session_history, input_messages_key="question", history_messages_key="chat_history")</pre>	
Memory strategy	Decides how much history to keep: all messages, last few messages, a summary, or relevant facts only.

Basic Memory

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables import RunnableWithMessageHistory

prompt = ChatPromptTemplate.from_template(
    "You are an helping assistant. Please give naswer professioanlly\n"
    "Message History : {history}"
    "User Message : {input}"
)

promptChain = prompt | base_llm | StrOutputParser()

store = {}

def get_sessionHistory(session_id):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chatRunnable = RunnableWithMessageHistory(
    promptChain,
    get_sessionHistory,
    input_messages_key="input",
    history_messages_key="history"
)

session_id = "user123"

response = chatRunnable.invoke(
    {"input":userInput},
    config={"configurable":{"session_id":session_id}}
)
```

Summary Memory

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables import RunnableWithMessageHistory

prompt = ChatPromptTemplate.from_template(
    "Act as a helping assistant and give answer professionally.\n"
    "History : {history}\n"
    "User Message : {userMessage}"
)

promptChain = prompt | base_llm | StrOutputParser()

store={}

def get_sessionHistory(session_id):
    /*
        Write logic to implement the summary for history that can be get or set using
        Store[session_id] for particular session ids.
    */

messageRunnable = RunnableWithMessageHistory(
    promptChain,
    get_sessionHistory,
    input_messages_key="userMessage",
    history_messages_key="history"
)

session_id = "user123"
```

Conversation Token Buffer Memory

It saves messages, calculates their token count, then removes the oldest messages when the configured limit is exceeded.

```
from langchain.memory import ConversationTokenBufferMemory
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
```

```
model = ChatOpenAI(model="gpt-4o-mini")
```

```
# 2. Token memory
```

```
memory = ConversationTokenBufferMemory(
    llm=model,
    max_token_limit=1000,
    memory_key="chat_history",
    return_messages=True
)
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{question}")
])
```

```
# 4. Reusable chat function
```

```
def chat(question):
```

```
    # Load saved memory
```

```
    memory_data = memory.load_memory_variables({})
```

```
    # Add saved memory + current question into the prompt
```

```
    final_messages = prompt.format_messages(
        chat_history=memory_data["chat_history"],
        question=question
    )
```

```
    # Send prompt to LLM
```

```
    response = model.invoke(final_messages)
```

```
    # Save current question and LLM response in memory
```

```
    memory.save_context(
        {"input": question},
        {"output": response.content}
    )
```

```
    return response.content
```

```
# First question
```

```
print(chat("My name is Somnath."))
```

```
# Second question
```

```
print(chat("I work with SharePoint."))
```

```
# Memory helps model answer this follow-up question
```

```
print(chat("What technology do I work with?"))
```

SECTION 2 : CHAINS

A **chain** connects multiple steps into one reusable workflow. It can also include memory, a retriever (RAG), tools, and conditional logic. In current LangChain, the **Runnable / |** (pipe) operator is the standard way to build chains.

Main Components

Input	Data provided when invoking the chain, such as <code>{"topic": "LangChain"}</code>
Prompt	Creates the instruction sent to the model and replaces variables like <code>{topic}</code> .
Output parser	Converts the AI response into plain text, JSON, or a Python object.
Runnable	Any component that can run with <code>.invoke()</code> . Prompts, models, parsers, retrievers, and custom functions are all runnables.
Memory (optional)	Adds prior conversation history into the prompt.
Retriever (optional)	Finds relevant documents before the prompt is sent to the model—used in RAG.
RunnablePassthrough	<code>RunnablePassthrough()</code> passes the input forward without changing it .

```
from langchain_core.runnables import
RunnablePassthrough
passthrough = RunnablePassthrough()
print(passthrough.invoke("What is RAG?"))

OutPut
What is RAG?
```

Basic LLM Chain

A simple chain containing a prompt, a model, and usually an output parser.

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Components
model = ChatOpenAI(model="gpt-4o-mini")

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in beginner-friendly language."
)

output_parser = StrOutputParser()

# Create chain
chain = prompt | model | output_parser

# Run chain
response = chain.invoke({
    "topic": "LangChain"
})

print(response)
```

Sequential Chain

The output of one step becomes input for the next step.

```
summary_chain = (
    PromptTemplate.from_template("Summarize {topic} in one line.")
    | model
    | parser
)

summary = summary_chain.invoke({"topic": "RAG"})

detail_chain = (
    PromptTemplate.from_template("Explain this simply: {summary}")
    | model
    | parser
)

print(detail_chain.invoke({"summary": summary}))
```

SequentialChain

Runs multiple tasks at the same time using the same input.

```
from langchain_core.runnables import RunnableParallel

definition_chain = (
    PromptTemplate.from_template("Define {topic} in one line.")
    | model
    | parser
)

example_chain = (
    PromptTemplate.from_template("Give one example of {topic}.")
    | model
    | parser
)

parallel_chain = RunnableParallel(
    definition=definition_chain,
    example=example_chain
)

result = parallel_chain.invoke({"topic": "Embeddings"})

print(result["definition"])
print(result["example"])
```

Conditional/Router Chain

Chooses a different prompt or chain depending on the input.

```
from langchain_core.runnables import RunnableBranch

technical_chain = (
    PromptTemplate.from_template("Explain technically: {question}")
    | model
    | parser
)

simple_chain = (
    PromptTemplate.from_template("Explain simply: {question}")
    | model
    | parser
)

router_chain = RunnableBranch(
    (
        lambda data: data["type"] == "technical",
        technical_chain
    ),
    simple_chain
)

print(router_chain.invoke({
    "type": "technical",
    "question": "What is a vector database?"
}))
```

RAG Chain

Retrieves relevant documents first, then gives those documents as context to the LLM.

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.prompts import ChatPromptTemplate

# Assume retriever is already created from your vector database
def format_docs(docs):
    return "\n".join(doc.page_content for doc in docs)

prompt = ChatPromptTemplate.from_template("""
Answer only from this context:

{context}

Question: {question}
""")

rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough()
    }
    | prompt
    | model
    | parser
)

print(rag_chain.invoke("What is the leave policy?"))
```

SECTION 3 : RUNNABLES (build on the LCEL chain from Section 2)

A Runnable is any LangChain component that can accept input, perform work, and return output.

Main Runnable components

Input	Data provided to the runnable, such as a string or dictionary.
Transformation	The work performed: format prompt, call LLM, retrieve document, or run Python logic.
Output	Result returned after processing.
.invoke()	Runs one input synchronously.
.ainvoke()	Runs one input asynchronously using await.
.batch()	Runs multiple inputs together.
.stream()	Returns output gradually, useful for streaming LLM tokens.
operator	Connects runnables in sequence.

RunnableLambda

Converts a normal Python function into a runnable.

```
from langchain_core.runnables import RunnableLambda

add_exclamation = RunnableLambda(
    lambda text: text + "!"
)

print(add_exclamation.invoke("Hello"))
```

RunnableSequence

Runs steps one after another. The first output becomes the second input.

```
from langchain_core.runnables import RunnableLambda

remove_spaces = RunnableLambda(lambda text: text.strip())
uppercase = RunnableLambda(lambda text: text.upper())

sequence = remove_spaces | uppercase

print(sequence.invoke(" langchain "))
```

RunnableParallel

Sends the same input to multiple runnables at the same time and returns a dictionary.

```
from langchain_core.runnables import RunnableLambda,
RunnableParallel
```

```
uppercase = RunnableLambda(lambda text: text.upper())
length = RunnableLambda(lambda text: len(text))
```

```
parallel = RunnableParallel(
    upper_text=uppercase,
    text_length=length
)
```

```
print(parallel.invoke("langchain"))
```

RunnableBranch

Chooses a different runnable depending on a condition. It works like an if/else statement.

```
from langchain_core.runnables import RunnableLambda,
RunnableBranch

positive = RunnableLambda(lambda number: "Positive number")
negative_or_zero = RunnableLambda(lambda number: "Negative or
zero")

branch = RunnableBranch(
    (lambda number: number > 0, positive),
    negative_or_zero
)

print(branch.invoke(10))
print(branch.invoke(-5))
```

RunnableWithMessageHistory

RunnableWithMessageHistory adds **automatic chat memory** to an existing chain.

```
Chat = RunnableWithMessageHistory(
    #existing prompt | model chain
    chain
    #function that returns stored history
    get_session_history,
    #User input
    input_messages_key="question",
    #Placeholder in prompt for history
    history_messages_key="chat_history"
)
```

Hierarchy: PROMPTS define the template → MEMORY fills its history slot → CHAINS combine prompt+LLm+memory into one callable → RUNNABLES are the LCEL bricks ("|") that compose all of the above.